

Quick start: the Compliance Surveillance Engineering Team

Version 0.37.0. One page: the mental model, the three steps to a first engagement, the commands worth knowing, and the three safety rules that are enforced in code. The rendered copies (`docs/quick-start.html` , `docs/quick-start.pdf`) are produced from this file; edit here.

How it works

Virtual Surv-IT is a Claude Code plugin. **Morgan**, the PM agent, is the one front door: she classifies what you ask for (a problem, a review, or something to build), decides how many specialists the job needs and says why, briefs them, and brings the results back to you at each gate. Thirteen specialists do the work, each with one job: builders who write code and specs, and reviewers who can only read and advise. **Every piece of work is checked by a different agent than the one that produced it** before it counts as done, and a human signs off the last step.

- **A loop is an iteration cycle, not a failure.** Findings go back for a fix and are re-reviewed. Morgan says plainly when something is not finished.
-  **observed** and  **inferred** mark every data claim: what was checked against evidence versus what was reasoned. An inference is never presented as fact.
- **Every engagement gets its own folder** under `VSIT/engagements/<slug>` in your project: brief, work, tests, QA evidence, review findings, delivery report, summary email and a generated START-HERE index. Nothing lives only in the chat.

Three steps

1. **Install** (from a terminal, not inside Claude Code): `git clone https://github.com/danieledge/virtual-surv-IT.git && cd virtual-surv-IT && python install_helper.py`. The helper checks prerequisites (Python 3.9+, `git`, the `claude` CLI; on Windows, Git Bash or WSL for the hooks), runs the real `claude` plugin

`marketplace add` and `claude plugin install` commands, and optionally fetches pinned, SHA-256-verified review analysers (`--no-downloads` fetches nothing).

2. **Enable it per project.** From the project you want the team in, run `/plugin` and enable **compliance-surveillance-team** for that project (or `virt-surv configure`). Not at user scope: every enabled plugin's agent roster loads into every session.
3. **Start.** Restart Claude Code in that project and type `/compliance-surveillance-team:demo` to watch a narrated engagement on synthetic data, or `/compliance-surveillance-team:engage <what you need>`. After that, reply in plain English; Morgan stays in role for the session. Recommended launch: `virt-surv go` from the project folder shows the project's settings and pre-seeds the resume-or-new choice.

The commands worth knowing

`/engage` reaches everything; the others are shortcuts. Type `/` in Claude Code to see the full set, or read `docs/operating-guide.d/command-index.md`.

When	Command	What it does
Have a look first	<code>virt-surv try</code>	one minute, zero tokens: a closed synthetic review engagement with its evidence room, opened in your browser (<code>python scripts/try_engagement.py</code> from the clone)
	<code>/demo</code>	the whole team on synthetic data, every decision narrated
	<code>/team</code>	who the 13 specialists are and what each owns (<code>--preferences</code> , <code>--dashboard</code> for the utilities)
Before any real data	<code>/prepare-data</code>	safe data first: synthetic, or masked by approved tooling. Its own masking is a best-effort aid, not an anonymisation pipeline; masked output is still personal data
The everyday work	<code>/review</code>	a proper review of existing code, before a PR or an audit (<code>--depth</code> , <code>--focus</code> , <code>--fix</code>)
	<code>/build --scenario</code>	a detection scenario from spec through build to sign-off
	<code>/analyse-data</code>	a question you want evidenced, not just answered
	<code>/why-no-alert</code>	why a case did not alert, or a scenario went silent
	<code>/engage --light</code>	a small, non-regulated fix without the full ceremony
Finishing	<code>/handover</code>	hand real code to a dev or ops team, with the QA evidence

Three safety rules, enforced in code

1. **Raw data never reaches the model.** Anything under your project's raw-data folder is blocked by an always-on guard hook, in every session, whether or not the team is engaged. Use synthetic data or data masked by approved means.
2. **The code under review never runs without a human's consent marker.** Reviews are static by default. Running tests, the script or a profiler needs `.claude/.exec-consent` ,

which only you can create; Morgan is blocked from writing it and will not ask you to work around that. The team's own vendored scripts run consent-free.

3. **Nothing is done until the gate says so.** Closing an engagement runs a mechanical Definition-of-Done check (artifacts present, rendered, QA and reviews recorded, summary email written) and refuses while anything is open. Human sign-off is the one item only you can complete.

The guards are strong controls on the file tools and a real deterrent on the shell, but on the shell they are lexical rather than a sandbox; `docs/safety-model.md` states exactly what each one guarantees.

What people ask first

- **Will it change my code without asking?** Builders write files; reviewers cannot edit anything, by tool grant. Work lands in the engagement's own folder unless you ask for it in your source tree, and every change is reviewed before the engagement closes.
- **What does it cost?** Morgan states the specialist count and the reason before any fan-out; the status line shows the running spend. A narrated review demo is light; a full build delivery is a real multi-agent run (see the README's token-usage section).
- **Where does my data go?** Every file the model reads goes to the model provider. That is why raw data is blocked outright and why anything else needs your attestation that it carries no prohibited PII or MNPI.
- **Can I stop half way?** Yes. The state on disk records what was done and what is outstanding; `/engage` offers to resume it next time.